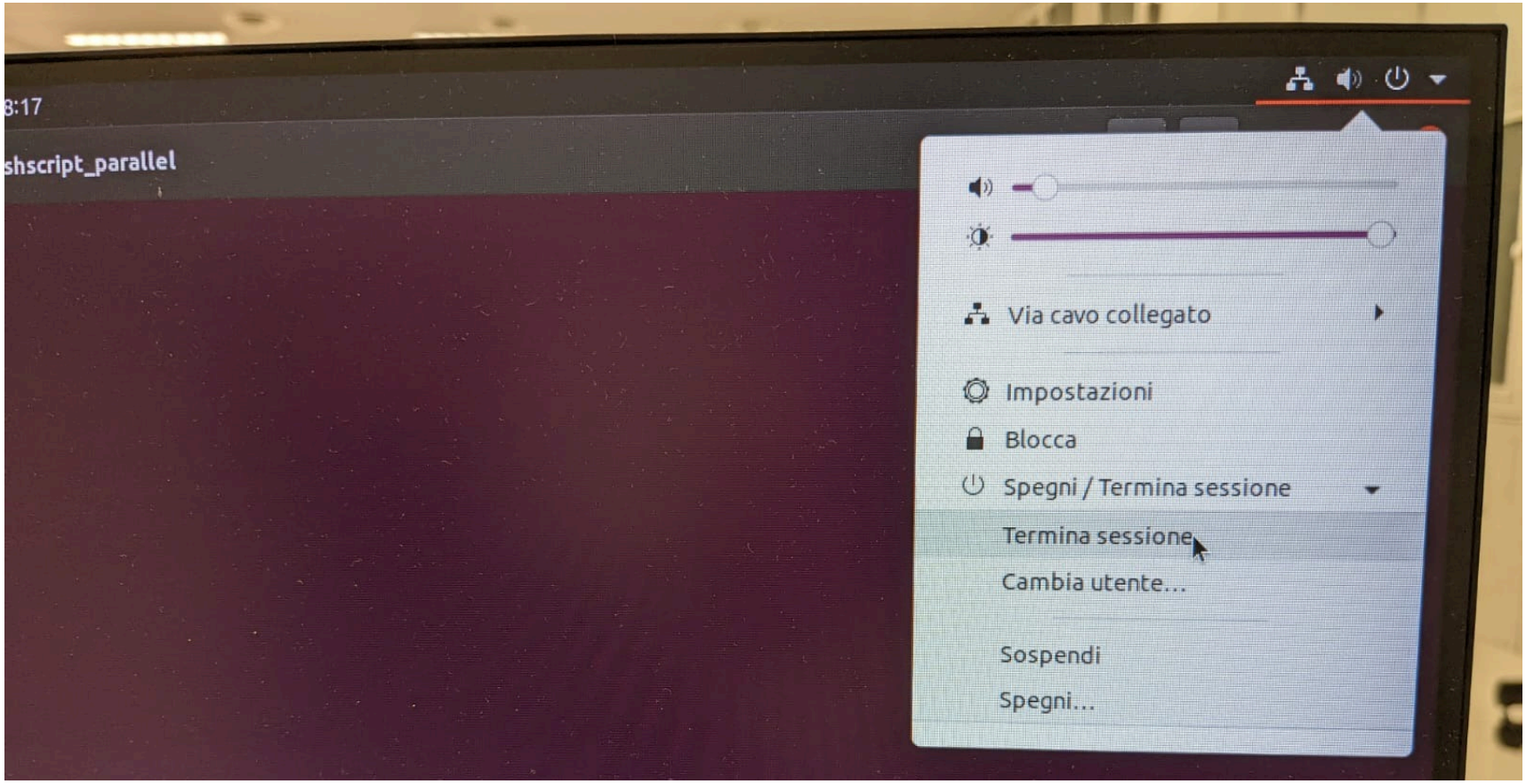
Corso di Sistemi Operativi

Corso estinto di Sistemi Operativi e Reti - Modulo Sistemi Operativi

Prova scritta - Febbraio 2026

LEGGI ATTENTAMENTE LE ISTRUZIONI:

1. **Rinomina** subito la cartella chiamata "Cognome-Nome-Matricola" che hai trovato sul Desktop e in cui hai trovato questa traccia, sostituendo "Cognome" "Nome" e "Matricola" con i tuoi dati personali e **lasciando i trattini senza spazi**; se hai un doppio nome oppure un doppio cognome dovrai chiamare la cartella come in questo esempio:
    a. DeLuca-MarcoGiovanni-199999
2. **Carica** tutto il materiale didattico che vorrai usare sul Desktop; puoi farlo solo nei primi 5 minuti della prova;
3. **Svolgi** il compito; lascia tutto il sorgente che hai prodotto nella cartella di cui al punto 1;
4. Quando hai finito fai "**Termina Sessione/Logout**", ma lascia la postazione mantenendo il PC acceso.
5. **E' tua diretta responsabilità** garantire l'integrità del tuo elaborato, anche in caso di assenza di corrente. **Salva spesso il tuo lavoro**



e NON spegnere il PC.

SALVA SPESSO

CI SONO DEI PUNTI AMBIGUI NELLA TRACCIA? DECIDI TU COSA FARE

È parte integrante di questo esercizio completare le specifiche date nei punti non esplicitamente definiti, introducendo nuove strutture dati, o estendendo quelle preesistenti laddove si ritenga necessario, risolvendo eventuali ambiguità. Si può cambiare il codice dei metodi esistenti dove serve.

POSSO CAMBIARE IL PROTOTIPO DEI METODI RICHIESTI O DI QUELLI ESISTENTI? Si, ma solo se li lasci retrocompatibili

Il codice che scriverai è pensato per essere usato come un modulo di libreria. Un modulo di libreria potrebbe essere usato da altri programmatori, i quali si aspettano di trovare una specifica interfaccia.

Se quindi modifichi il prototipo dei metodi pubblici di una classe, devi assicurarti che possano essere comunque utilizzati per come erano stati pensati prima delle tue modifiche. Puoi aggiungere qualsivoglia campo e metodo privato, e qualsivoglia classe ausiliaria. Analogamente, i metodi esistenti possono essere modificati nel loro codice, ma non se ne deve cambiare il risultato finale o il significato a meno che non ti venga richiesto esplicitamente.

CHE LINGUAGGIO POSSO USARE? PYTHON 3.X

POSSO CONSENTIRE SITUAZIONI DI RACE CONDITION NEL MIO CODICE? NO
POSSO CONSENTIRE SITUAZIONI DI DEADLOCK NEL MIO CODICE? NO
POSSO CONSENTIRE ALTRE SITUAZIONI DI BLOCCO TOTALE NEL MIO CODICE, TIPO NESTED LOCKOUT, LIVELOCK O ALTRO? NO
POSSO CONSENTIRE SITUAZIONI DI STARVATION NEL MIO CODICE? SI, tranne quando ti viene chiesto esplicitamente di rimuoverle
MA IL MAIN() LO DEVO AGGIORNARE? E I THREAD DI PROVA? SI

E' obbligatorio implementare esplicitamente il codice di prova oppure modificare il codice di prova pre-esistente, e accertarsi che giri senza errori prima della consegna.

ESERCIZIO 1 - PROGRAMMAZIONE MULTITHREADED

(Punteggio minimo richiesto 18/30. Pesa per ⅔ del voto finale)

Svolgi tutti i punti della traccia in un unico file che chiamerai `SOLUZIONE.py`
`-> Qualsiasi altro nome di file verrà IGNORATO <-`

Punto 1

Fai in modo che l'operazione `download_pezzettone` possa casualmente fallire nel 10% dei casi. Quando il download di un pezzettone fallisce, il worker corrispondente lascia perdere il pezzettone su cui è avvenuto il fallimento, rimette quest'ultimo nella task queue, e prova con un altro, sempre fornito dalla task queue stessa.

Punto 2

Introduci il metodo `aggiungi_worker(self,n)` che avvia dinamicamente `n` worker per contribuire alle operazioni di download, e il metodo `rimuovi_worker(self,n)` che arresta fino a `n` worker, ma garantendo che almeno un worker resti sempre attivo. *Un worker che viene arrestato deve terminare l'iterazione corrente per poi uscire.*

Punto 3

Introduci la possibilità di mettere in pausa il download. Dovrà esserci un metodo `pause` che sospende temporaneamente tutte le operazioni di download. Il metodo `restart` invece deve consentire di riprendere le operazioni di download da dove erano state interrotte. Per "sospendere" si intende che i worker terminano l'iterazione corrente per poi sospendere temporaneamente di scaricare pezzettoni, e per "riprendere" si intende che i worker ricominciano a scaricare pezzettoni, a partire dallo stato del download precedente.

SALVA SPESSO

ESERCIZIO 2 - LINGUAGGI DI SCRIPTING

(Punteggio minimo richiesto 18/30. Pesa per ⅓ del voto finale)

Trovi allegato a questa traccia uno script Perl che presenta errori di sintassi e/o logici. Sistemalo per ottenere il comportamento che ti descriviamo sotto. ***NON devi stravolgere il contenuto dello script o riscriverlo da capo: la sua struttura deve rimanere pressoché invariata. E' prevista una penalità se correggi parti di codice che invece non presenta bug.***

Lo script `path_feb26.pl` dovrebbe accettare da riga di comando, come unico argomento, una stringa rappresentante il nome di un comando. Il programma deve cercare se tale comando è presente in una delle directory elencate nella variabile d'ambiente `$PATH`. Se presente, deve inoltre verificare che l'utente proprietario abbia i diritti di esecuzione.
Se il file viene trovato e soddisfa le suddette condizioni:

- Stampare a video il messaggio "`<cmd> trovato in <percorso_directory_contenente_il_comando>`".
- Terminare immediatamente l'esecuzione.

Se al termine della ricerca in tutte le directory il comando non viene trovato (o non si hanno i permessi richiesti), stampare il messaggio: "`Il comando <cmd> non è presente nella variabile d'ambiente $PATH`".

Ad esempio, se lo script venisse invocato come

```
perl path_feb26.pl sudo
```

e il path `/usr/bin` fosse presente nella variabile `$PATH`, allora lo script dovrebbe stampare

```
sudo trovato in /usr/bin
```

Qualora invece il path `/usr/bin` non fosse in `$PATH`, allora lo script dovrebbe stampare

```
Il comando sudo non è presente nella variabile d'ambiente $PATH
```